Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Graph Neural Networks for Constructing (k, g)-Graphs

Bachelor Thesis

2026
Vladimír Jančár

Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Graph Neural Networks for Constructing (k, g)-Graphs

Bachelor Thesis

Study Programme: Applied Computer Science
Field of Study: Computer Science
Department: Department of Applied Informatics
Supervisor: Mgr. Ján Pastorek

Bratislava, 2026
Vladimír Jančár

Acknowledgments:

This section will be completed near the end of the thesis.

1 Abstract

This thesis investigates the use of graph neural networks as learned heuristics for constructing small regular graphs of prescribed girth, also known as (k, g)-graphs. The final abstract will summarize the problem, proposed method, experiments, and main findings once the experimental contribution is fixed.

Keywords: graph neural networks, algebraic graph theory, cage problem, (k, g)-graphs, voltage graph lifts, heuristic search

Contents

List of Figures

List of Tables

2 Introduction

Temporary purpose of this chapter. This chapter should be written after the core contribution is clear, but before the abstract and conclusion. It should introduce the cage problem, explain why constructing small (k, g)-graphs is difficult, and state the thesis goal: using graph neural networks as learned heuristics inside constrained graph-construction search.

This thesis concerns the construction of (k, g)-graphs: k-regular graphs whose shortest cycle has length g. The classical cage problem asks for such graphs with the smallest possible order. Known approaches include algebraic constructions, voltage graph lifts, exhaustive search, heuristic search, and excision techniques [1], [2].

The working hypothesis of this thesis is that graph neural networks are most useful not as unrestricted graph generators, but as learned components inside constrained search procedures. Regularity and girth can often be enforced, checked, or pruned directly, while a learned model can score partial states, voltage assignments, or local graph edits.

To write later. Add the motivation, exact research questions, thesis contribution, and a paragraph describing the final chapter structure.

3 Background and Definitions

Temporary purpose of this chapter. This should be one of the first substantial chapters to draft. Definitions are the foundation for the rest of the thesis, and writing them early exposes notation problems before experiments and results are described.

3.1 Graphs

A graph is a pair $G = (V, E)$, where V is a finite set of vertices and E is a set of edges. In the main cage-construction setting, graphs are simple, undirected graphs unless explicitly stated otherwise.

For a vertex $v \in V$, the degree of v, denoted $\deg(v)$, is the number of edges incident with v.

3.2 Regular Graphs

A graph is k-regular if every vertex has degree k. Regularity is one of the two hard constraints in the (k, g)-graph construction task.

3.3 Walks, Paths, and Cycles

A walk is a sequence of vertices and edges in which consecutive vertices are adjacent. A path is a walk with no repeated vertices. A cycle is a closed path of length at least 3.

The girth of a graph, denoted $\mathrm{girth}(G)$, is the length of its shortest cycle. If a graph has no cycles, its girth is sometimes treated as infinite.

3.4 (k, g)-Graphs and Cages

A (k, g)-graph is a k-regular graph whose girth is at least g. A (k, g)-cage is a (k, g)-graph with the smallest possible number of vertices. This minimum order is commonly denoted $n(k, g)$.

The Moore bound gives a general lower bound on the order of a (k, g)-graph. It is useful both as a theoretical benchmark and as a practical target size for construction algorithms.

3.5 Graph Neural Networks

Graph neural networks process graph-structured data by iteratively updating vertex representations through message passing. In each layer, a node aggregates information from its neighbors and combines the aggregate with its current hidden representation [3].

The project uses several GNN architectures:

- GCN, a stable baseline based on degree-normalized neighbor aggregation.
- GraphSAGE, an inductive architecture based on learned aggregation.
- GIN, a sum-aggregation architecture with expressivity related to the Weisfeiler–Leman test [4].
- GPS, a graph transformer architecture combining local message passing with global attention.
- Loopy GNN, a cycle-aware architecture relevant to minimum-cycle and girth-related tasks.

3.6 Voltage Graph Lifts

A voltage graph construction starts with a small base graph, a finite group, and group-element labels on the directed edges. The corresponding lift expands the base graph into a larger graph. If the base graph is k-regular, the lifted graph is also k-regular.

This representation is useful for cage construction because it enforces regularity structurally and shifts the search problem toward choosing good voltage assignments.

To write later. Add exact Moore bound formulas, formal voltage-lift notation, and the net-voltage characterization of girth used by `ai/cage/voltage/cycle_analysis.py`.

4 Related Work

Temporary purpose of this chapter. This chapter should be drafted after the core definitions are stable. It should explain what classical graph theory and computational search already provide, so the thesis contribution is positioned honestly.

The cage problem has been studied through algebraic constructions, finite geometries, Cayley graphs, voltage graph lifts, exhaustive generation, local search, and excision.

The Dynamic Cage Survey provides the broad background and record-holder context [1].

Recent computational work is especially relevant to this thesis. Exoo et al. describe lift-based generation, pruning of voltage assignments, tabu search, hill climbing, and excision methods for improving upper bounds on cage and related graph-construction problems [2]. Exoo also studies order-decreasing transformations and graph-distance moves that preserve regularity and girth or make excision possible [5].

To write later. Separate this into subsections: finite-geometry/algebraic constructions, voltage graph lifts, exhaustive and heuristic search, excision, and learned heuristics. Be careful to cite every construction and not overstate novelty.

5 Preparatory GNN Tasks

Temporary purpose of this chapter. This chapter should explain why degree prediction and minimum-cycle prediction were useful stepping stones before cage construction. It should not become the main thesis contribution unless the final experiments make that necessary.

The project first evaluates GNNs on two prediction tasks: node degree prediction and minimum-cycle prediction. Degree prediction is local and exactly verifiable. It tests whether a model and training pipeline can recover a simple structural quantity. Minimum-cycle prediction is harder because cycle membership and shortest-cycle length depend on non-local graph structure.

The documentation and experiments currently indicate that GIN and GraphSAGE perform strongly on degree prediction, while GCN and Loopy are less suitable for that local task. For minimum-cycle prediction, small models struggle, and larger Loopy models improve substantially, which is consistent with the need for cycle-aware capacity.

To write later. Add dataset generation details, exact model configurations, metrics, and final result tables from the trained model registry. Keep this chapter subordinate to the cage-construction contribution.

6 Cage Construction Methods

Temporary purpose of this chapter. This is likely the central methodology chapter. It should explain the attempted construction methods, the search environments, the reward design, and the later voltage-lift direction.

The first construction approach models cage generation as sequential graph editing. The agent adds or removes edges while respecting constraints related to degree, connectivity, and girth. A direct guided-search approach was considered first, but the documentation identifies dataset construction for useful partial states as a bottleneck.

The reinforcement-learning approach uses PPO to learn from interaction rather than a fixed dataset. The reward design went through several iterations. Sparse rewards were not sufficient. Progress-based reward shaping improved the learning signal by measuring movement toward regularity and the target number of edges. A discounted shaping term created an unintended penalty for remaining near high-potential states, so the final version uses an undiscounted potential difference.

The second construction direction uses voltage graph lifts. In this setting, the agent or search procedure assigns voltages to the edges of a small base graph. The lifted graph is automatically regular when the base graph is regular, so the search can focus more directly on girth. The current codebase includes exact net-voltage girth computation, random search, brute force for small groups, GNN-guided beam search, and an RL environment for voltage assignment.

To write later. Decide which method becomes the main contribution: direct graph-editing PPO, voltage-assignment search, GNN-guided tabu search, or excision/ completion. Then document the final algorithm precisely enough to reproduce it.

7 Experiments and Results

Temporary purpose of this chapter. This should be written late, after experiments are stable. It should report final numbers, not exploratory anecdotes, and it should compare against non-learned baselines.

The current project documentation reports that PPO-based cage generation improved after reward shaping and larger models. A GPS model with hidden dimension 128, eight attention heads, and GIN convolution is currently described as the strongest direct graph-editing model, with nontrivial success rates on early curriculum stages.

For the final thesis, the results should answer whether the learned component improves construction search. The most important comparisons are likely:

- random search versus learned search,
- brute force on small cases where exhaustive search is possible,
- beam search versus GNN-guided beam search,
- tabu or hill-climbing baselines if implemented,
- direct graph editing versus voltage-assignment search.

To write later. Add final tables only after rerunning or verifying metrics. Avoid using stale documentation numbers as final thesis claims unless they are tied to exact model IDs, checkpoints, and evaluation scripts.

8 Conclusion

Write this last. The conclusion should be written after all experiments and result tables are final. It should state whether the thesis goal was achieved, summarize the main findings, describe limitations, and give concrete future work.

The final conclusion will summarize what was implemented and evaluated, whether graph neural networks provided useful guidance for constructing (k, g)-graphs, and which search formulation proved most promising.

To write later. Do not introduce new literature or new experiments here. The conclusion should only synthesize what the thesis has already shown.

Bibliography

[1] G. Exoo and R. Jajcay, "Dynamic Cage Survey," *The Electronic Journal of Combinatorics*, 2013.

[2] G. Exoo, J. Goedgebeur, J. Jooken, L. Stubbe, and T. Van den Eede, "New small regular graphs of given girth: the cage problem and beyond." 2025.

[3] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural Message Passing for Quantum Chemistry," in *Proceedings of the 34th International Conference on Machine Learning*, 2017.

[4] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?," in *International Conference on Learning Representations*, 2019.

[5] G. Exoo, "On decreasing the orders of (k, g)-graphs", *Journal of Combinatorial Optimization*, 2023, doi: 10.1007/s10878-023-01092-9.